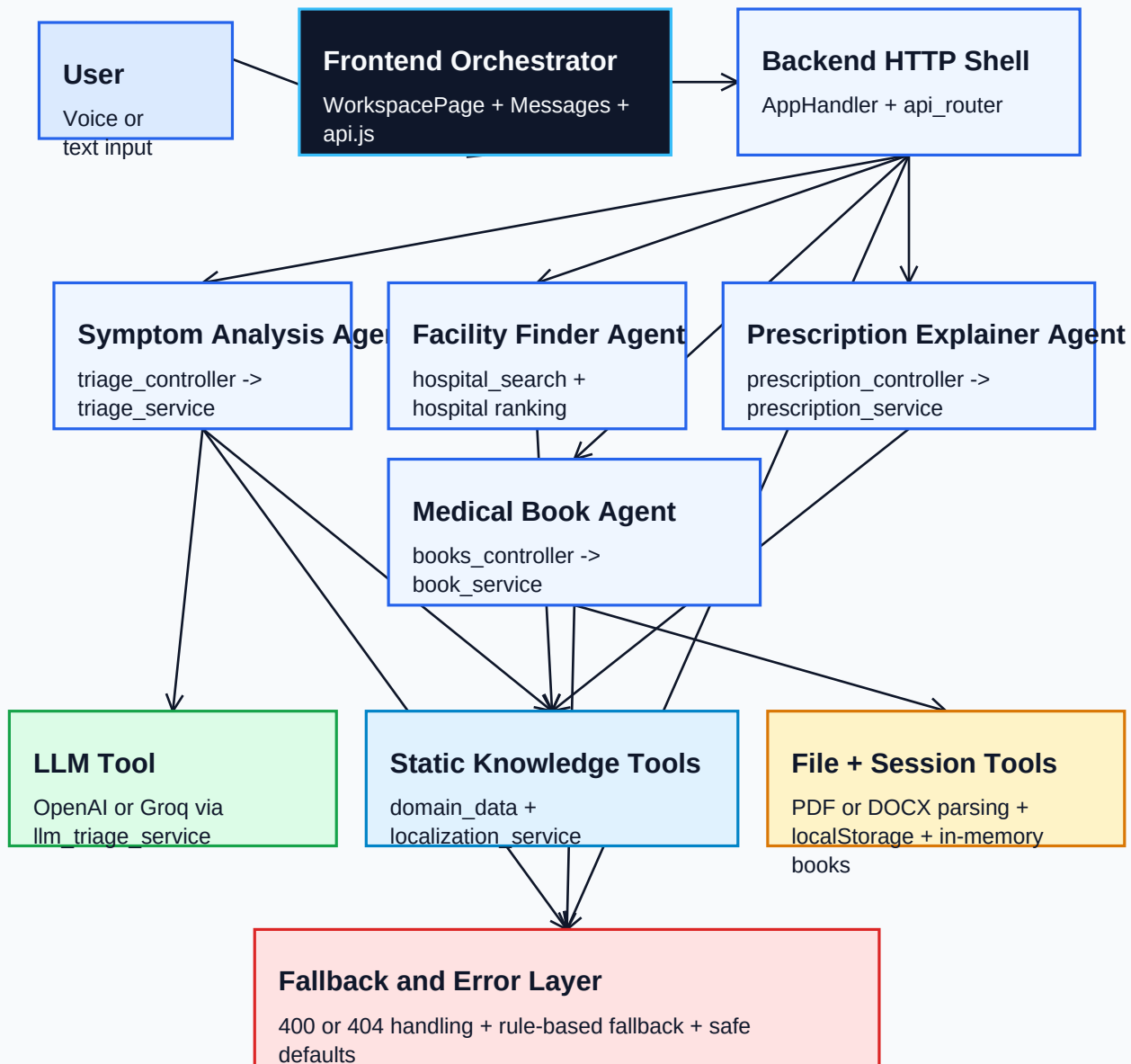


GenAi Agent Architecture

A 1-2 page system view of agent roles, communication, tool integrations, and failure handling

GenAi behaves like a focused multi-agent healthcare assistant behind one chat interface. The frontend orchestrates requests, the backend routes them to specialist agents, and each agent can call local tools or external providers while degrading safely when a dependency fails.

System view: orchestration, specialist agents, tools, and fallback paths



What this diagram shows

The frontend orchestrator is the single user-facing hub. Each backend capability acts like a specialist agent. External providers such as OpenAI or Groq are optional tools rather than mandatory dependencies.

Role, Tool, and Failure Notes

Condensed handoff summary for implementation, review, or presentation use

Agent roles

- Frontend Conversation Orchestrator: collects intent, routes requests, and renders structured replies.
- Symptom Analysis Agent: detects symptoms, scores urgency, suggests facilities, and can enrich explanations with OpenAI or Groq.
- Facility Finder Agent: ranks nearby hospitals using district, state, specialty, and urgency.
- Prescription Explainer Agent: decodes shorthand such as OD, BD, TDS, AC, and PC into readable schedules.
- Medical Book Agent: ingests uploaded documents, checks source trust, retrieves excerpts, and answers only from the provided text.

Error handling

- requestJson normalizes backend failures into readable UI errors.
- Invalid service input raises ValueError and becomes a 400 JSON response.
- Unknown routes return 404 and static path traversal is blocked with 403.
- If OpenAI or Groq is missing, unavailable, or returns malformed output, symptom analysis falls back to the built-in rule-based engine.
- If a medical-book answer is missing, the app states that the information is not available instead of hallucinating.

Communication

1. User message enters the chat UI through WorkspacePage and Messages.
2. frontend/src/services/api.js chooses the matching backend endpoint.
3. AppHandler and api_router dispatch the request to a focused controller.
4. The controller calls one service module, which may use domain data, file parsers, localization, or an external LLM.
5. Structured JSON returns to the frontend and is rendered as the next assistant message.

Tool integrations

- OpenAI or Groq chat completions for optional symptom explanation enhancement.
- domain_data.py for symptoms, hospitals, facilities, and public-health reference topics.
- localization_service.py for multilingual labels and copy fallback.
- PyMuPDF plus DOCX/XML and text parsers for uploaded medical books.
- localStorage for browser-side session state and in-memory backend storage for uploaded books.

Design summary

GenAi is intentionally built around graceful degradation. If a premium tool fails, the app still answers through built-in rules, local retrieval, and explicit safety messaging. That keeps the user experience stable while allowing the architecture to grow into a more capable agent platform over time.